

Cell 1 — Setup & Load

```
import pandas as pd
import numpy as np
import plotly.express as px
import plotly.graph_objects as go

pd.set_option('display.max_columns', None)

df = pd.read_csv('../global_superstore.csv')
print(f"Loaded: {df.shape[0]:,} rows, {df.shape[1]} columns")
df.head()
```

Loaded: 51,290 rows, 27 columns

	Category	City	Country	Customer ID	Customer Name	Discount	Market	ji_lu-shu	Order Date	Order ID	Order Priority	Product ID	Product Name	Profit	Quantity	Region	Row ID	Sales	Segment	Ship Date
0	Office Supplies	Los Angeles	United States	LS-172304	Lycoris Saunders	0.0	US	1	00:00.0	CA-2011-130813	High	OFF-PA-10002005	Xerox 225	9.3312	3	West	36624	19	Consumer	00:00.0
1	Office Supplies	Los Angeles	United States	MV-174854	Mark Van Huff	0.0	US	1	00:00.0	CA-2011-148614	Medium	OFF-PA-10002893	Wirebound Service Call Books, 5 1/2" x 4"	9.2928	2	West	37033	19	Consumer	00:00.0
2	Office Supplies	Los Angeles	United States	CS-121304	Chad Sievert	0.0	US	1	00:00.0	CA-2011-118962	Medium	OFF-PA-10000659	Adams Phone Message Book, Professional, 400 Me...	9.8418	3	West	31468	21	Consumer	00:00.0
3	Office Supplies	Los Angeles	United States	CS-121304	Chad Sievert	0.0	US	1	00:00.0	CA-2011-118962	Medium	OFF-PA-10001144	Xerox 1913	53.2608	2	West	31469	111	Consumer	00:00.0
4	Office Supplies	Los Angeles	United States	AP-109154	Arthur Pritchep	0.0	US	1	00:00.0	CA-2011-146969	High	OFF-PA-10002105	Xerox 223	3.1104	1	West	32440	6	Consumer	00:00.0

Cell 2 — Structure & Data Types

```
df.info()
```

<class 'pandas.DataFrame'>
RangeIndex: 51290 entries, 0 to 51289
Data columns (total 27 columns):
Column Non-Null Count Dtype
--- ---
0 Category 51290 non-null str
1 City 51290 non-null str
2 Country 51290 non-null str
3 Customer ID 51290 non-null str
4 Customer Name 51290 non-null str
5 Discount 51290 non-null float64
6 Market 51290 non-null str
7 ji_lu-shu 51290 non-null int64
8 Order Date 51290 non-null str
9 Order ID 51290 non-null str
10 Order Priority 51290 non-null str
11 Product ID 51290 non-null str
12 Product Name 51290 non-null str
13 Profit 51290 non-null float64
14 Quantity 51290 non-null int64
15 Region 51290 non-null str
16 Row ID 51290 non-null int64
17 Sales 51290 non-null int64
18 Segment 51290 non-null str
19 Ship Date 51290 non-null str
20 Ship Mode 51290 non-null str
21 Shipping Cost 51290 non-null float64
22 State 51290 non-null str
23 Sub-Category 51290 non-null str
24 Year 51290 non-null int64
25 Market2 51290 non-null str
26 weeknum 51290 non-null int64
dtypes: float64(3), int64(6), str(18)
memory usage: 19.6 MB

Cell 3 — Nulls & Duplicates Check

```
print("Missing values per column:")
print(df.isnull().sum()[df.isnull().sum() > 0] if df.isnull().sum().sum() > 0 else "None – dataset is clean")

print(f"\nDuplicate rows: {df.duplicated().sum()}")
```

Missing values per column:
None – dataset is clean

Duplicate rows: 0

Cell 4 — Numerical Summary

```
df[['Sales', 'Profit', 'Discount', 'Quantity', 'Shipping Cost']].describe().round(2)
```

	Sales	Profit	Discount	Quantity	Shipping Cost
count	51290.00	51290.00	51290.00	51290.00	51290.00
mean	246.50	28.61	0.14	3.48	26.38
std	487.57	174.34	0.21	2.28	57.30
min	0.00	-6599.98	0.00	1.00	0.00
25%	31.00	0.00	0.00	2.00	2.61
50%	85.00	9.24	0.00	3.00	7.79
75%	251.00	36.81	0.20	5.00	24.45
max	22638.00	8399.98	0.85	14.00	933.57

Cell 5 — Categorical Attribute Overview

```
categorical_cols = ['Category', 'Sub-Category', 'Segment', 'Region', 'Market',
                    'Market2', 'Ship Mode', 'Order Priority']

for col in categorical_cols:
    print(f"\n{col} ({df[col].nunique()} unique):")
    print(df[col].value_counts().head(6))
```

```
Category (3 unique):
Category
Office Supplies    31273
Technology          10141
Furniture           9876
Name: count, dtype: int64
```

```
Sub-Category (17 unique):
Sub-Category
Binders      6152
Storage      5059
Art          4883
Paper        3538
Chairs       3434
Phones       3357
Name: count, dtype: int64
```

```
Segment (3 unique):
Segment
Consumer     26518
Corporate    15429
Home Office   9343
Name: count, dtype: int64
```

```
Region (13 unique):
Region
Central      11117
South        6645
EMEA         5029
North        4785
Africa       4587
Oceania      3487
Name: count, dtype: int64
```

```
Market (7 unique):
Market
APAC         11002
LATAM        10294
EU           10000
US           9994
EMEA         5029
Africa       4587
Name: count, dtype: int64
```

```
Market2 (6 unique):
Market2
APAC         11002
North America 10378
LATAM        10294
EU           10000
EMEA         5029
Africa       4587
Name: count, dtype: int64
```

```
Ship Mode (4 unique):
Ship Mode
Standard Class 30775
```

Cell 6 — Spatial Coverage

```
print(f"Countries: {df['Country'].nunique()}")
print(f"States: {df['State'].nunique()}")
print(f"Cities: {df['City'].nunique()}")
print(f"\nMarket distribution:\n{df['Market'].value_counts()}")
```

```
Countries: 147
States: 1094
Cities: 3636

Market distribution:
Market
APAC         11002
LATAM        10294
EU           10000
US           9994
EMEA         5029
Africa       4587
Canada       384
Name: count, dtype: int64
```

Cell 7 — Temporal Fields: Investigating Order Date / Ship Date

```
# Order Date and Ship Date arrived corrupted in this export (text-only, no real dates)
print("Order Date unique values:", df['Order Date'].unique())
print("Ship Date unique values:", df['Ship Date'].unique())

# Year and weeknum are intact and will substitute for time-series analysis
print("\nYear range:", df['Year'].min(), "-", df['Year'].max())
print("Year distribution:\n", df['Year'].value_counts().sort_index())
print("\nweeknum range:", df['weeknum'].min(), "-", df['weeknum'].max())
```

```
Order Date unique values: <ArrowStringArray>
['00:00.0']
Length: 1, dtype: str
Ship Date unique values: <ArrowStringArray>
['00:00.0']
Length: 1, dtype: str

Year range: 2011 - 2014
Year distribution:
Year
2011     8998
2012    10962
2013    13799
2014    17531
Name: count, dtype: int64

weeknum range: 1 - 53
```

Cell 8 — Cleaning

```
# ji_lu-shu is a constant column (always 1) – no analytical value
print("ji_lu-shu unique values:", df['ji_lu-shu'].unique())
df = df.drop(columns=['ji_lu-shu'])

# Build a usable pseudo-date from Year + weeknum since Order Date/Ship Date are unusable
df['Year_Week'] = df['Year'].astype(str) + '-W' + df['weeknum'].astype(str).str.zfill(2)
df['Period'] = pd.to_datetime(df['Year'].astype(str) + df['weeknum'].astype(str).str.zfill(2) + '1', format='%Y%W%w')

print(f"\nCleaned shape: {df.shape}")
df[['Year', 'weeknum', 'Year_Week', 'Period']].head()
```

ji_lu-shu unique values: [1]

Cleaned shape: (51290, 28)

	Year	weeknum	Year_Week	Period
0	2011	2	2011-W02	2011-01-10
1	2011	4	2011-W04	2011-01-24
2	2011	32	2011-W32	2011-08-08
3	2011	32	2011-W32	2011-08-08
4	2011	40	2011-W40	2011-10-03

Cell 9 — Save Cleaned Dataset

```
df.to_csv('global_superstore_cleaned.csv', index=False)
print(f"Saved cleaned dataset: {df.shape[0]:,} rows, {df.shape[1]} columns")
```

Saved cleaned dataset: 51,290 rows, 28 columns

Start coding or [generate](#) with AI.